

Lec18

Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm that organizes code into **objects** that contain both **data (properties)** and **behavior (methods)**.

Instead of writing everything as separate variables and functions, OOP allows us to model real-world entities such as Students, Cars, Employees, Bank Accounts, etc.

Why do we need OOP?

Without OOP

```
let student1Name = "A";  
let student1Age = 20;
```

```
let student2Name = "B";  
let student2Age = 21;
```

As the number of students increases, the code becomes difficult to manage.

With OOP

```
let student = {  
  name: "A",  
  age: 20  
};
```

Now data and related actions stay together.

Benefits

Code Reusability

Better Organization

Easy Maintenance

Scalability

Real-world Modeling

Core Concepts of OOP

Objects

Classes

Constructors

Methods

Encapsulation

Inheritance

Polymorphism

Abstraction

Objects

An object is a collection of properties and methods.

Example

A Car has:

Properties:

- Brand
- Model
- Color

Behaviors:

- Start
- Stop
- Accelerate

```
const car = {  
  brand: "Toyota",  
  model: "Camry",  
  color: "Black",  
  
  start() {  
    console.log("Car Started");  
  }  
};
```


Problem with Objects

Suppose we need 100 cars.

```
const car1 = {...};
```

```
const car2 = {...};
```

```
const car3 = {...};
```

We would repeat code many times.

To solve this problem, JavaScript provides **Classes**.

Classes

A Class is a blueprint or template for creating objects.

Think of:

- Class = Building Blueprint
- Object = Actual Building

Example

Class = Student

Every student has:

- name
- age
- course

Every student can:

- study()
- attendClass()

Constructor Method

A constructor initializes object values.

It automatically runs when an object is created.

```
class Student {
```

```
    constructor(name, age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

```
    }
```

```
}
```

Understanding this

this refers to the current object being created.

For example:

```
this.name = name;
```

means:

Store the received name inside the current object's name property.

Creating Objects from Class

Use new keyword.

```
const student1 = new Student("John", 20);  
const student2 = new Student("Emma", 21);
```

Output

```
console.log(student1);
```

Output:

```
Student {  
  name: "John",  
  age: 20  
}
```

Adding Methods

```
class Student {  
  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  study() {  
    console.log(` ${this.name} is studying`);  
  }  
  
}
```


Using Methods

```
const student1 = new Student("A", 20);
```

```
student1.study();
```

Output:

A is studying

Multiple Objects from Same Class

```
const s1 = new Student("A", 20);  
const s2 = new Student("B", 21);  
const s3 = new Student("C", 19);
```

All share the same blueprint.

Memory advantage of Classes

Without classes:

```
const student1 = {  
  study() {}  
};
```

```
const student2 = {  
  study() {}  
};
```

The same method gets duplicated.

With classes:

```
class Student {  
  study() {}  
}
```

Method is shared among all objects.

This saves memory.

Inheritance

Inheritance allows one class to acquire properties and methods of another class.

Example

Animal

- eat()
- sleep()

Dog

- bark()

Cat

- meow()

Dog and Cat both can eat and sleep.

Instead of rewriting code, we inherit.

Parent Class

```
class Animal {  
  
  eat() {  
    console.log("Eating");  
  }  
  
  sleep() {  
    console.log("Sleeping");  
  }  
  
}
```

Child Class

```
class Dog extends Animal {
```

```
  bark() {
```

```
    console.log("Barking");
```

```
  }
```

```
}
```

Using Inherited Methods

```
const dog = new Dog();
```

```
dog.eat();
```

```
dog.sleep();
```

```
dog.bark();
```

Output:

Eating

Sleeping

Barking

Constructor in Inheritance

Suppose Animal has properties.

```
class Animal {  
  
    constructor(name) {  
        this.name = name;  
    }  
  
}
```

Child class:

```
class Dog extends Animal {  
  
    constructor(name, breed) {  
        super(name);  
        this.breed = breed;  
    }  
  
}
```


What is super()?

`super ( )` calls the parent constructor.

Without it:

```
class Dog extends Animal {  
  
    constructor(name, breed) {  
        this.breed = breed;  
    }  
  
}
```

Error occurs.

Correct:

```
super(name);
```

Example

```
const dog = new Dog("Tommy", "Labrador");
```

```
console.log(dog);
```

Output:

```
Dog {  
  name: "Tommy",  
  breed: "Labrador"  
}
```

Method Overriding

A child class can replace the parent's method.

Parent

```
class Animal {  
  
    makeSound() {  
        console.log("Animal Sound");  
    }  
  
}
```

Child

```
class Dog extends Animal {  
  
    makeSound() {  
        console.log("Bark Bark");  
    }  
  
}
```

```
const dog = new Dog();
```

```
dog.makeSound();
```

Output:

Bark Bark

This is called Method Overriding.

Example

Parent Class

```
class Employee {  
  
    constructor(name, salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    work() {  
        console.log(`${this.name} is working`);  
    }  
  
}
```

Child Class

```
class Developer extends Employee {  
  
    code() {  
        console.log(`${this.name} is coding`);  
    }  
  
}
```

```
const dev = new Developer("Naman", 50000);
```

```
dev.work();
```

```
dev.code();
```

Output:

Naman is working

Naman is coding

Task

Single Inheritance

Multiple Inheritance

Multilevel Inheritance

Hierarchical Inheritance

Hybrid Inheritance